

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	J KARTHIK	Reg. No.:	192472136
Section / Batch:	CSE – AI / 2024	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I J KARTHIK / 192472136 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK

Q1. 0/1 Knapsack Optimization

Problem Analysis

This problem requires selecting a subset of items, each usable at most once, to maximize total profit within a fixed weight capacity. Because each item introduces a binary choice (include or exclude) and the capacity constraint creates overlapping subproblems (the same remaining capacity can be reached through different item combinations), a brute-force approach would take exponential time. A Dynamic Programming approach is appropriate since the problem exhibits both optimal substructure (the optimal solution for capacity w can be built from optimal solutions of smaller capacities) and overlapping subproblems, which DP solves by storing intermediate results in a table.

Aim

To determine the maximum profit obtainable by selecting a combination of items, without splitting or reusing any item, such that the total weight does not exceed the given knapsack capacity, using Dynamic Programming.

Algorithm

1. Start.
2. Read the number of items n , the profit array values[], the weight array weights[], and the knapsack capacity W .
3. Create a 2D table $dp[n+1][W+1]$ and initialize the first row and first column to 0.
4. For each item i from 1 to n , and for each capacity w from 0 to W :
 - a. If $weights[i-1] \leq w$, set $dp[i][w] = \max(values[i-1] + dp[i-1][w - weights[i-1]], dp[i-1][w])$.
 - b. Otherwise, set $dp[i][w] = dp[i-1][w]$.
5. The value $dp[n][W]$ gives the maximum achievable profit.
6. Print the maximum profit.
7. Stop.

Python Code

```
def knapsack01(n, values, weights, W):
    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            if weights[i - 1] <= w:
```

```

        dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],
                        dp[i - 1][w])
    else:
        dp[i][w] = dp[i - 1][w]

    return dp[n][W]

n = 3
values = [60, 100, 120]
weights = [10, 20, 30]
W = 50

print("Maximum Profit =", knapsack01(n, values, weights, W))

```

Input

```

n = 3
Values = 60 100 120
Weights = 10 20 30
W = 50

```

Output

Maximum Profit = 220

Complexity Analysis

Time Complexity: $O(n \times W)$ — the DP table has $(n+1)$ rows and $(W+1)$ columns, and each cell is filled in constant time.

Space Complexity: $O(n \times W)$ for the full 2D table; this can be optimized to $O(W)$ by using a single 1D array and iterating capacity from W down to $\text{weights}[i-1]$.

Q11. Longest Common Subsequence (LCS)

Problem Analysis

The task is to find the length of the longest subsequence shared by two strings while preserving relative character order. A naive recursive solution would re-solve the same (i, j) index pairs repeatedly, since the recursion branches at every mismatch, giving exponential time. Because the problem has overlapping subproblems (the same suffix pairs of s1 and s2 recur across branches) and optimal substructure (the LCS of two strings can be derived from the LCS of their prefixes), it is a natural fit for Dynamic Programming using a 2D table indexed by string positions.

Aim

To determine the length of the longest subsequence common to two given strings, where the subsequence need not be contiguous but must preserve the relative order of characters, using Dynamic Programming.

Algorithm

1. Start.
2. Read the two strings s1 and s2 of lengths m and n respectively.
3. Create a 2D table $dp[m+1][n+1]$ and initialize the first row and first column to 0.
4. For each i from 1 to m and each j from 1 to n:
 - a. If $s1[i-1]$ equals $s2[j-1]$, set $dp[i][j] = dp[i-1][j-1] + 1$.
 - b. Otherwise, set $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.
5. The value $dp[m][n]$ gives the length of the longest common subsequence.
6. Print the LCS length.
7. Stop.

Python Code

```
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i - 1] == s2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
```

```
return dp[m][n]
```

```
s1 = "ABCDGH"
```

```
s2 = "AEDFHR"
```

```
print("LCS Length =", lcs(s1, s2))
```

Input

```
s1 = ABCDGH
```

```
s2 = AEDFHR
```

Output

LCS Length = 3

Complexity Analysis

Time Complexity: $O(m \times n)$ — where m and n are the lengths of $s1$ and $s2$; every cell of the DP table is computed once in constant time.

Space Complexity: $O(m \times n)$ for the full 2D table; can be reduced to $O(\min(m, n))$ if only the length (not the actual subsequence) is required, by keeping just two rows at a time.

Q21. Activity Selection Problem

Problem Analysis

The goal is to pick the maximum number of non-overlapping activities from a set of activities with given start and finish times. Unlike Knapsack or LCS, this problem does not require exploring all combinations — a locally optimal choice (always picking the activity that finishes earliest among the remaining feasible ones) leads to a globally optimal solution. This greedy-choice property, combined with optimal substructure, makes a Greedy strategy both correct and more efficient than a Dynamic Programming table-based approach for this problem.

Aim

To select the maximum number of non-overlapping activities from a given set of activities, each having a start time and a finish time, using a Greedy approach.

Algorithm

1. Start.
2. Read the number of activities n , along with their `start[]` and `finish[]` times.
3. Sort all activities in increasing order of their finish time.
4. Select the first activity in the sorted order and mark its finish time as `last_finish`; set `count = 1`.
5. For each subsequent activity, if its start time is greater than or equal to `last_finish`, select it, increment `count`, and update `last_finish` to its finish time.
6. Repeat step 5 for all remaining activities.
7. The value of `count` gives the maximum number of non-overlapping activities.
8. Print the result.
9. Stop.

Python Code

```
def activity_selection(start, finish):  
    n = len(start)  
    activities = sorted(zip(start, finish), key=lambda x: x[1])  
  
    count = 1  
    last_finish = activities[0][1]  
  
    for i in range(1, n):  
        if activities[i][0] >= last_finish:  
            count += 1
```

```
        last_finish = activities[i][1]

    return count

start = [1, 3, 0, 5]
finish = [2, 4, 6, 7]

print("Max Activities =", activity_selection(start, finish))
```

Input

```
n = 4
start = 1 3 0 5
finish = 2 4 6 7
```

Output

Max Activities = 3

Complexity Analysis

Time Complexity: $O(n \log n)$ — dominated by sorting the activities by finish time; the single subsequent scan takes $O(n)$.

Space Complexity: $O(n)$ for storing the sorted list of (start, finish) pairs; $O(1)$ additional space is used for the count and last_finish tracking variables.

Q42. Fibonacci with Memoization

Problem Analysis

The plain recursive definition of the Fibonacci sequence recomputes the same subproblems many times — for example, `fibonacci(n-2)` is evaluated independently within both the `fibonacci(n-1)` and `fibonacci(n)` branches — which causes the naive recursion to grow exponentially. Since the subproblems overlap heavily and each depends only on the two preceding results, storing already-computed values (memoization) eliminates redundant work. This makes Dynamic Programming (top-down memoization) the appropriate technique.

Aim

To compute the *n*th Fibonacci number efficiently by storing and reusing previously computed values, thereby avoiding redundant recursive calculations, using Dynamic Programming (Memoization).

Algorithm

1. Start.
2. Read the value of *n*.
3. Create a dictionary `memo` to store already-computed Fibonacci values.
4. Define a recursive function `fibonacci(n)`:
 - a. If $n \leq 1$, return *n*.
 - b. If *n* is already present in `memo`, return `memo[n]`.
 - c. Otherwise, compute `memo[n] = fibonacci(n-1) + fibonacci(n-2)` and return `memo[n]`.
5. Call `fibonacci(n)` and store the result.
6. Print the computed Fibonacci value.
7. Stop.

Python Code

```
def fibonacci(n, memo={}):  
    if n <= 1:  
        return n  
    if n in memo:  
        return memo[n]  
  
    memo[n] = fibonacci(n - 1, memo) + fibonacci(n - 2, memo)  
    return memo[n]
```



```
n = 10  
print("Fibonacci =", fibonacci(n))
```

Input

n = 10

Output

Fibonacci = 55

Complexity Analysis

Time Complexity: $O(n)$ — each value from 0 to n is computed and stored exactly once due to memoization, instead of the $O(2^n)$ of naive recursion.

Space Complexity: $O(n)$ — used by the memo dictionary and by the recursion call stack in the worst case.

Q56. Number of Ways to Climb Stairs

Problem Analysis

The number of ways to reach step n , taking 1 or 2 steps at a time, depends only on the number of ways to reach the two preceding steps ($n-1$ and $n-2$), since those are the only positions from which step n is directly reachable. This overlapping-subproblem structure — identical to the Fibonacci recurrence — means repeated recursive calls would recompute the same step counts many times. Building the solution bottom-up with Dynamic Programming avoids this redundancy and computes the answer in linear time.

Aim

To determine the total number of distinct ways to reach the top of a staircase of n steps, given that a person can climb either 1 or 2 steps at a time, using Dynamic Programming.

Algorithm

1. Start.
2. Read the number of steps n .
3. If $n \leq 1$, return 1 as the only possible way.
4. Create an array $dp[0..n]$ and initialize $dp[0] = 1$ and $dp[1] = 1$.
5. For each i from 2 to n , compute $dp[i] = dp[i-1] + dp[i-2]$.
6. The value $dp[n]$ gives the total number of distinct ways to reach the top.
7. Print the result.
8. Stop.

Python Code

```
def climb_stairs(n):
    if n <= 1:
        return 1

    dp = [0] * (n + 1)
    dp[0], dp[1] = 1, 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

```
n = 4  
print("Ways =", climb_stairs(n))
```

Input

n = 4

Output

Ways = 5

Complexity Analysis

Time Complexity: $O(n)$ — the DP array is filled once from index 2 to n, with each cell computed in constant time.

Space Complexity: $O(n)$ for the DP array; can be optimized to $O(1)$ by keeping only the last two computed values instead of the full array.

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____